

Rapport de veille technologique Front-end - Vue.js

Ali Abakar Issa

1. Contexte et méthode

En tant que lead technique d'une start-up, je recommande **Vue.js** comme technologie front-end du nouveau produit. Ce rapport garde la trace de ce choix et de ce qui l'a motivé.

La démarche a combiné deux sources de connaissance, que je distingue tout au long du rapport :

Technologie	Mon niveau	Origine de mes affirmations
Vue	Pratiqué en profondeur (4 jours)	Code du Kanban, documentation officielle
React	Pratiqué en entreprise (2 ans d'alternance)	Expérience professionnelle, veille
Angular	Non pratiqué	Veille uniquement (documentation officielle)

Le Kanban Board réalisé en Vue (colonnes, tâches, édition, suppression, persistance locale) a servi de terrain d'essai. Les sources privilégiées sont primaires : dépôts GitHub, documentations officielles, sondages originaux. Toutes ont été consultées le 3 octobre 2026 et sont listées avec leurs limites en fin de rapport.

2. Présentation des trois technologies

Les trois technologies reposent sur des composants réutilisables, mais leur philosophie diffère.

React (Meta) est une bibliothèque d'interface. On décrit l'interface en JSX et l'état est immuable : chaque modification recrée des objets. Avantages : écosystème immense, premier marché de l'emploi, grande liberté d'architecture. Inconvénients : il faut assembler soi-même routage, état global et formulaires ; les règles des hooks et les rendus inutiles demandent de la rigueur ; React est le premier point de douleur cité dans State of JS 2025.

Angular (Google) est un framework complet et très structuré : TypeScript obligatoire, injection de dépendances, routage, formulaires et tests intégrés, réactivité par signals. Avantages : cadre homogène pour de grandes équipes, 24 mois de support par version, outil de migration officiel. Inconvénients : courbe d'apprentissage la plus raide, nombreux concepts avant d'être productif, plus verbeux pour un petit produit.

Vue (projet communautaire créé par Evan You) est un framework progressif : templates proches du HTML, réactivité automatique par proxies, outils officiels (Pinia, Vue Router, Vite). Avantages : prise en

main rapide, code concis, bon compromis entre liberté et cadre. Inconvénients : marché de l'emploi plus restreint, pas de calendrier de support fixe, écosystème moins vaste que celui de React.

	React	Angular	Vue
Nature	Bibliothèque	Framework complet	Framework progressif
Réactivité	Virtual DOM, état immuable	Signals, détection de changements	Proxies, Virtual DOM (Vapor Mode optionnel en 3.6)
Usage (Stack Overflow 2025)	44,7 %	18,2 %	17,6 %
Satisfaction « admired » (Stack Overflow 2025)	52,1 %	44,7 %	50,9 %
Support des versions	Correctifs de sécurité rétroportés	24 mois par majeure	Pas de calendrier fixe

3. Justification du choix de Vue

Je choisis Vue pour le produit de la start-up, pour quatre raisons.

Rapidité de mise sur le marché : en cinq jours, sans connaître Vue, j'ai livré toutes les fonctionnalités obligatoires. La réactivité automatique supprime une catégorie entière de bugs fréquents en React, comme les dépendances oubliées ou l'état modifié par erreur. Un fichier .vue regroupe template, logique et style.

Formation de l'équipe : les cinq semaines prévues suffisent largement. Un développeur qui connaît HTML et JavaScript lit un template Vue sans formation. Un développeur React retrouve ses repères avec la Composition API, très proche des hooks.

Pérennité : Vue 3 est stable depuis 2020 et publie des correctifs presque chaque semaine. La version 3.6, en cours de finalisation, reste compatible avec le code existant : son Vapor Mode promet des gains de performance sans réécriture.

Sobriété : deux dépendances en production, 35 Ko transférés, EcoIndex A.

J'assume deux faiblesses. Le recrutement est plus difficile qu'avec React, mais un profil React devient opérationnel sur Vue en quelques jours, comme je l'ai constaté. Angular aurait offert un cadre plus strict, mais il est surdimensionné pour une petite équipe qui doit livrer vite.

4. La philosophie de Vue, vue dans mon code

Vue repose sur une idée centrale : **l'état est réactif, on le modifie directement et Vue met l'interface à jour seul**. Techniquement, `ref()` enveloppe la donnée dans un Proxy JavaScript qui détecte chaque lecture et chaque écriture.

Apport : des mutations directes. Dans le store du Kanban, ajouter une tâche s'écrit en une ligne :

```
// stores/board.ts – Vue détecte le push grâce au Proxy
findColumn(columnId)?.tasks.push({ id: crypto.randomUUID(), title, description: '',
createdAt: new Date().toISOString() })
```

En React, la même action impose de recréer le tableau des colonnes et celui des tâches

(setColumns(cols => cols.map(...))), sous peine d'une interface qui ne se met pas à jour.

Apport : la persistance en trois lignes. Un `watch` profond surveille tout l'arbre de données et sauvegarde à chaque changement, où qu'il ait lieu :

```
watch(columns, (value) => saveBoard(value), { deep: true })
```

Flux de données. Comme en React, les données descendent par les props, en lecture seule. Différence : un enfant ne reçoit pas de fonction de rappel, il **émet un événement** (`emit('add', titre)`) que le parent écoute avec `@add`. Les templates ajoutent des directives concises : `v-model` pour la liaison bidirectionnelle d'un champ, `@submit.prevent`, `@keydown.esc`.

Limites rencontrées :

- Le `.value` obligatoire sur les `ref` dans le script, mais pas dans le template : une source d'oubli au début.
- Le DOM est mis à jour de façon asynchrone. Pour donner le focus à un champ qui vient d'apparaître, il faut attendre `await nextTick()`.
- La magie de la réactivité a un revers : une prop modifiée par erreur dans un enfant changerait le store. D'où le choix d'un **brouillon local** dans la modal d'édition, copié à l'ouverture et envoyé au store seulement à l'enregistrement.

5. Écosystème et pérennité

Vue est stable et actif : on reste sur la version 3.5 et on passera à 3.6 sans réécriture.

	Vue	Angular	React
Version stable (oct. 2026)	3.5.43 (17/09/2026)	22 (03/06/2026)	19.2.x
Prochaine étape	3.6 en Release Candidate (rc.9, 18/09/2026)	v23 vers juin 2027	Pas de majeure annoncée
Rythme	Pas de calendrier fixe, correctifs quasi hebdomadaires	1 majeure par an depuis v22	Majeures espacées de plusieurs années
Support	Branche courante suivie	12 mois actifs + 12 mois LTS	Correctifs de sécurité rétroportés

Ce qui arrive pour Vue. La version 3.6 apporte le **Vapor Mode**, un rendu sans Virtual DOM, et une réactivité réécrite. Le Vapor Mode est optionnel et ne couvre qu'une partie des API : le code du Kanban continuera de fonctionner tel quel.

Ce que ça implique pour maintenir le produit. On ne met pas une Release Candidate en production : le produit reste sur 3.5. Sans LTS formelle, l'équipe doit suivre les mises à jour mineures régulièrement, ce qui est peu coûteux puisqu'elles ne cassent pas le code au sein de Vue 3. Le précédent à retenir est la migration Vue 2 → Vue 3 (fin de vie de Vue 2 le 31/12/2023), qui a été difficile pour beaucoup d'équipes. Partir directement sur Vue 3 évite ce risque.

Une leçon de veille. Un article de blog publié en juillet 2026 annonçait Vue 3.6 « officiellement sorti ». Le dépôt GitHub officiel montre au contraire qu'elle est toujours en Release Candidate fin septembre. Les sources primaires priment sur les articles secondaires.

6. Architecture de l'application

L'application est une SPA en trois couches ; remplacer la source de données ne demande de modifier qu'un seul fichier.

Board

Veille Techno1

+ Add a task

Veille techno1

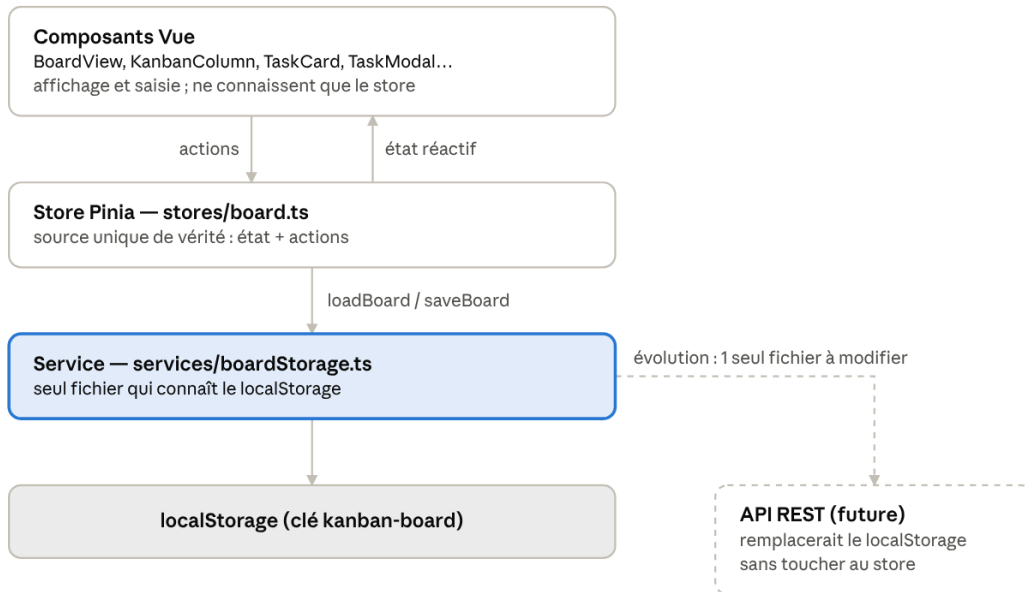
+ Add a task

Veille techno1

+ Add a task

+ Add a column

Seul le service de stockage connaît le localStorage



architecture du Kanban · 3 couches

Les composants ne parlent qu'au store, le store ne parle qu'au service, et seul le service connaît le localStorage. Passer à une API REST ne toucherait que `services/boardStorage.ts`.

Composants. `BoardView` est le seul à appeler le store. Les autres reçoivent des props et émettent des événements : `AddColumnForm`, `AddTaskForm`, `TaskCard` et `TaskModal` ne savent pas d'où viennent les données, ce qui les rend réutilisables. L'état d'interface (la tâche sélectionnée) reste local à `BoardView` ; le store ne contient que les données métier.

Modèle de données. Une colonne contient directement ses tâches (`Column { id, title, tasks: Task[] }`), ce qui évite de relier tâches et colonnes par un identifiant.

SPA ou rendu serveur ? Une SPA suffit ici : un outil interne, sans enjeu de référencement, sans backend. Le rendu serveur (Nuxt pour Vue, Next.js pour React, Angular SSR) se justifie pour un site public qui doit être indexé ou s'afficher vite sur mobile.

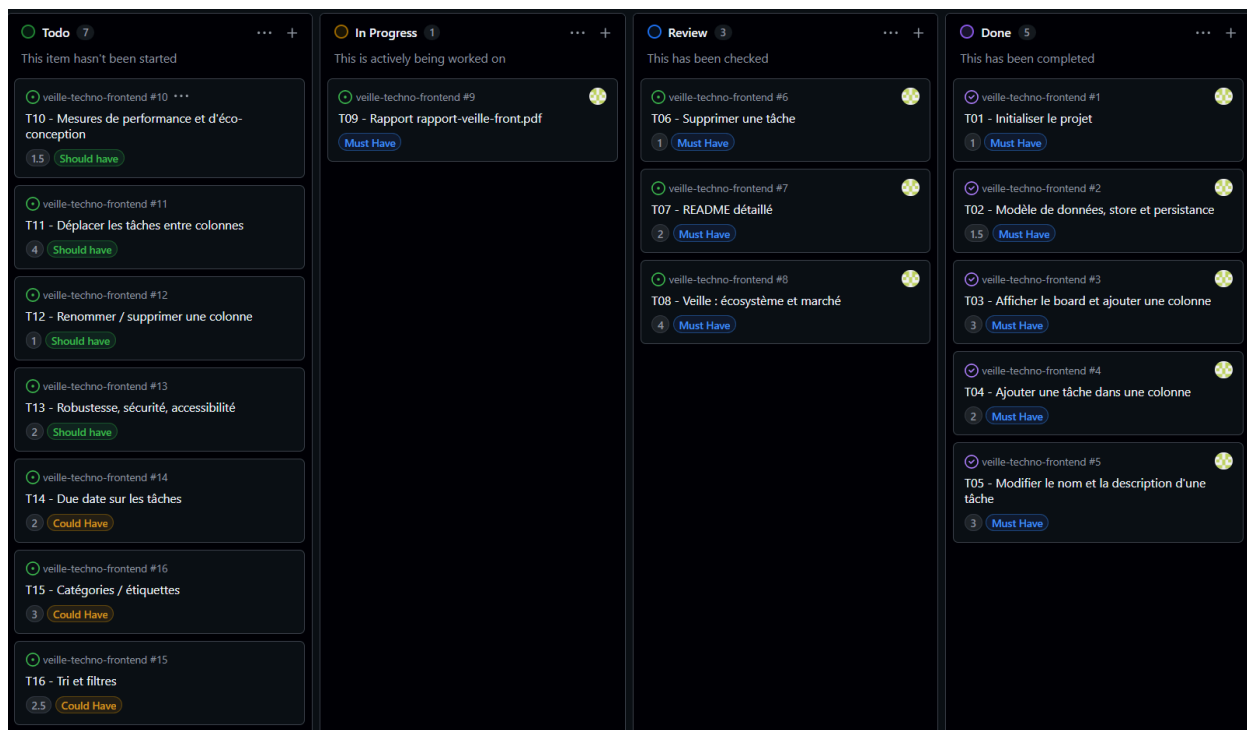
Framework complet ou bibliothèque ? Vue se situe entre les deux : le cœur est léger, et les briques officielles (Pinia, Vue Router) s'ajoutent au besoin. Ici, Pinia suffit ; aucun routeur n'est nécessaire pour une seule page.

7. Rétrospective : mener le projet en 5 jours

Les fonctionnalités obligatoires ont été livrées, mais les plus gros dépassements ne sont pas venus de Vue : ils sont venus du CSS et de l'environnement de mesure.

Découpage. Avant de coder, le travail a été découpé en 16 tickets sur GitHub Projects (user story, critères d'acceptation, priorité MoSCoW). En cours de route, l'accès au localStorage a été isolé dans un service, et T05 (édition) et T06 (suppression) ont été regroupés dans une même modal.

Priorisation. Critère principal : ce qu'exige la consigne. Avec environ une journée de retard le samedi, j'ai sacrifié le **drag & drop** (non exigé, estimé à 4 h, le ticket le plus incertain) ainsi que les bonus (date d'échéance, filtres, catégories), pour protéger le rapport et la soutenance, eux exigés.



Estimations et temps réel.

Ticket	Estimation	Réel	Écart	Cause principale
T01 Initialisation	1 h (a posteriori)	1 h	0	Options de create-vue mal cochées, projet recréé
T02 Store + persistance	1 h 30 (a posteriori)	2 h 30	+1h	Regarde un tuto
T03 Board + colonnes	2 h	4 h	+2 h	Bug de défilement horizontal (CSS)
T04 Ajout de tâche	1 h 30	1 h 30	0	—

Ticket	Estimation	Réel	Écart	Cause principale
T05 + T06 Édition et suppression	3 h	2 h 30	-30 min	<dialog> natif plus simple que prévu
T10 Mesures	30 min	3 h	+2 h 30	Cache, dist/ non régénéré, robots.txt manquant

Les écarts, reliés à la courbe d'apprentissage :

- **T03, un piège CSS, pas Vue.** Une barre de défilement apparaissait sur toute la page. Diagnostic par la console : un `<label>` masqué (`sr-only`, en position absolue) n'avait aucun ancêtre positionné et étirait la page. Corrigé par une seule classe, `relative`. J'ai d'abord suspecté le plugin Vue DevTools, à tort.
- **T04, un concept Vue.** Le focus sur un champ qui vient d'apparaître ne marchait pas : Vue met le DOM à jour de façon asynchrone. Solution : `await nextTick()`.
- **T10, l'environnement de mesure.** Première mesure EcoIndex faussée par le cache (0 requête), modifications invisibles faute de rebuild, `robots.txt` absent. Les mesures ont pris six fois le temps prévu.

Ma courbe d'apprentissage, comparée à React. Ce qui m'a semblé familier : les composants, les props, `v-for` et sa `key`, la Composition API proche des hooks. Ce qui m'a surpris positivement : modifier l'état directement, sans copier les objets, et ne plus gérer de tableaux de dépendances. Ce qui m'a dérouté : `.value`, les événements émis au lieu des callbacks, et le DOM asynchrone.

Pour l'équipe. Un développeur React sera productif en quelques jours. Ce qui freinera la montée en compétence : les subtilités de la réactivité (`.value`, `nextTick`, ne pas modifier les props), à couvrir dès la première semaine de formation.

8. Un environnement sobre : dépendances, mesures, arbitrages

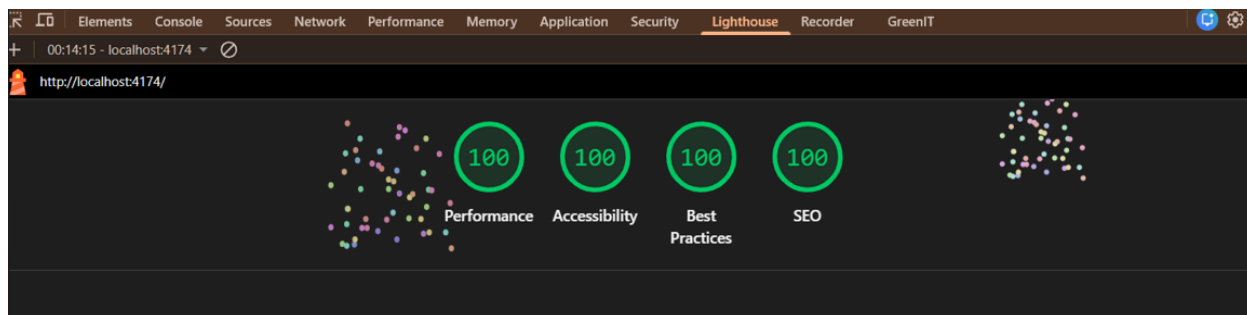
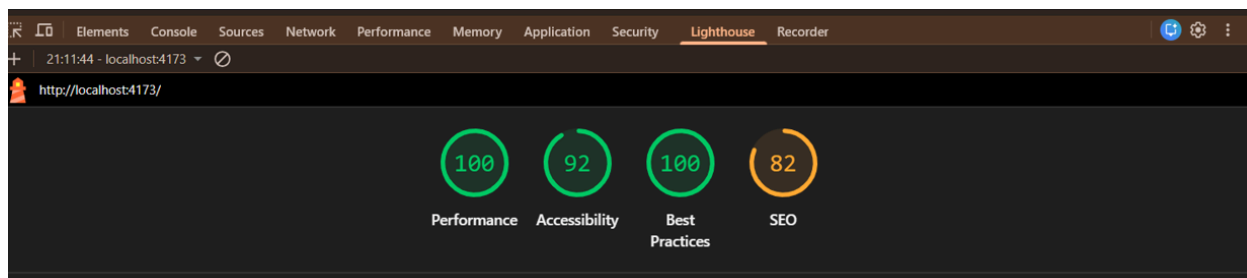
L'application transfère 34 Ko en 3 requêtes, obtient 100 aux quatre scores Lighthouse et un EcoIndex A.

Deux dépendances en production seulement, `vue` et `pinia`. Tailwind, TypeScript et les outils de qualité ne servent qu'au développement. Trois besoins sont couverts par des API natives du navigateur plutôt que par une bibliothèque : la modal (`<dialog>`), les identifiants (`crypto.randomUUID()`) et la confirmation de suppression (`confirm()`).

Mesures sur le build de production (`npm run build` puis `npm run preview`, navigation privée) :

Mesure	Première mesure	Après corrections
Lighthouse : Performance / Accessibilité / Bonnes pratiques / SEO	100 / 92 / 100 / 82	100 / 100 / 100 / 100

Les corrections : attribut lang sur <html>, meta description, fichier robots.txt, styles d'impression.
Une première mesure EcoIndex (97,87, 0 requête) a été écartée : elle était faussée par le cache du navigateur.



Arbitrages assumés :

- **CSS bloquant le rendu (150 ms estimées).** Le corriger demanderait un outil d'extraction du CSS critique. Avec une performance déjà à 100, une dépendance de plus ne se justifie pas.
- **3 bonnes pratiques GreenIT non validées sur 18.** En-têtes de cache et HTTP/2 dépendent du serveur, pas de l'application : le serveur local de prévisualisation sert en HTTP/1.1 sans cache. Les noms de fichiers hachés (index-Bip1Fb1Y.js) permettent déjà un cache long en production. La feuille d'impression existe (@media print généré par Tailwind) mais l'outil ne la détecte pas ; ajouter un fichier séparé pour le satisfaire coûterait une requête de plus.
- **Le drag & drop natif** plutôt qu'une bibliothèque, s'il est ajouté plus tard : même logique de sobriété.

L'éco-conception n'a pas décidé du choix de Vue : les trois technologies permettent une application légère. Elle a en revanche guidé chaque dépendance.

9. Recommandation finale

Je m'engage sur Vue 3 (branche 3.5, avec Pinia et Vite) pour le produit de la start-up.

Enjeu de la start-up	Vue	Ce qui pèse
Rapidité de mise sur le marché	Favorable	Fonctionnalités obligatoires livrées en 5 jours sans expérience préalable

Enjeu de la start-up	Vue	Ce qui pèse
Formation de l'équipe	Favorable	Templates proches du HTML ; Composition API proche des hooks React
Pérennité	Favorable, sous condition	Projet actif, évolution 3.6 compatible ; mais pas de LTS, mises à jour régulières à prévoir
Recrutement	Défavorable	Vue à 17,6 % d'usage contre 44,7 % pour React (Stack Overflow 2025)

Marché de l'emploi. Je n'ai pas pu réaliser de relevé chiffré fiable : Mais d'après mon expérience de recherche d'alternance et les offres consultées, React et Angular sont les deux frameworks les plus demandés en France, Vue.js nettement moins. Un relevé manuel le même jour sur deux ou trois sites serait nécessaire pour le quantifier.

Le risque de recrutement est réel mais maîtrisé : on peut recruter des profils React et les former, ce que ma propre expérience montre faisable en quelques jours.

Ce qui me ferait changer d'avis :

- **Un recrutement massif et rapide** (plus de 10 développeurs en quelques mois) : je passerais à React pour le volume de candidats.
- **Une équipe déjà Angular, ou un produit très gros et réglementé** (banque, administration) : Angular, pour son cadre strict et son support de 24 mois.
- **Un besoin fort de référencement ou de rendu serveur** : Vue resterait valable, mais avec Nuxt plutôt qu'une SPA seule.
- **Un ralentissement du projet Vue** (correctifs qui s'espacent, 3.6 qui ne sort pas, écosystème qui s'étiole) : je réévaluerais lors de la veille suivante.

10. Sources et limites

Toutes les sources ont été consultées le 3 octobre 2026.

Source	Type	Utilisée pour	Limites
GitHub vuejs/core - Releases	Primaire	Versions de Vue, état de la 3.6	Aucune sur les faits ; ne dit rien de la feuille de route au-delà de 3.6
Angular - Versioning and releases	Primaire	Version, rythme et support d'Angular	Dates futures indicatives

Source	Type	Utilisée pour	Limites
State of JavaScript 2025	Sondage original (environ 13 000 répondants, fin 2025)	Tendances d'usage, points de douleur	Public auto-sélectionné, plutôt passionné et anglophone ; ne mesure pas l'emploi
Stack Overflow Developer Survey 2025 (chiffres via cette compilation)	Sondage original (environ 49 000 répondants, mi-2025)	Usage et satisfaction	Déclaratif, utilisateurs de Stack Overflow, données de 2025
versionlog.com - React	Secondaire	Politique de support de React	Agrégateur, à recouper avec react.dev
versio.io - React	Secondaire	Dernière version de React	Agrégateur commercial
ecorpit.com - Vue.js en 2026	Secondaire	Calendrier de l'écosystème Vue	Article d'une entreprise de services
jobrise.io - React, Vue, Angular et l'emploi	Secondaire	Tendance qualitative du marché français	Aucun chiffre ; à compléter par un relevé direct
Mesures Lighthouse et GreenIT-Analysis	Mesure directe (4 octobre 2026)	Performance, accessibilité, EcoIndex	Serveur local, une seule machine